

El Patrón Invocador-Ejecutor: Mitigando la Monopolización de Hardware y las Fugas de Memoria en el Aprendizaje Profundo Distribuido a Gran Escala

William Steve Rodriguez Villamizar (wisrovi rodriguez)

AI Leader & Solutions Architect

wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Resumen y Palabras Clave

Resumen: Los cuellos de botella clásicos en PyTorch y arquitecturas YOLO—tales como procesos zombi, fugas de memoria severas y una deficiente gestión de la memoria compartida—hacen colapsar rutinariamente clústeres enteros de GPUs durante optimizaciones a gran escala. Presentamos la arquitectura “Invoker-Celery” para mitigar estos estados de fallo. Al desacoplar explícitamente la orquestación de tareas de la ejecución física, utilizamos contenedores Docker efímeros (Executors) limitados por restricciones estrictas de hardware (`nano_cpus`, `mem_limit`, y `shm_size`). Esto asegura un aislamiento físico y lógico absoluto. Cuando ocurre un pico de memoria severo, el contenedor se destruye instantáneamente, dejando el demonio persistente (Invoker) intacto. Nuestros estudios de ablación empírica muestran que este aislamiento reduce los fallos críticos de nodos de un promedio de 18 por día a cero, eliminando la necesidad de reinicios físicos manuales y garantizando casi un 100 % de estabilidad operativa.

Palabras Clave: Aprendizaje Profundo Distribuido, Colas de Tareas Celery, Contenedores Efímeros, Aislamiento de Hardware, YOLO.

2. Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el eco-

sistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3. Introducción

El aprendizaje profundo distribuido es fundamentalmente susceptible a la degradación del nodo anfitrión con el tiempo. A medida que los clústeres escalan para procesar cargas de trabajo masivas de visión por computadora, los investigadores frecuentemente encuentran caídas silenciosas por falta de memoria (Out-Of-Memory u OOM) donde el demonio principal de entrenamiento asigna memoria más allá de los límites físicos de la GPU. En canales tradicionales de MLOps distribuido, un intermediario centralizado distribuye estas tareas de entrenamiento a un demonio persistente (como un worker de Celery) que ejecuta el bucle de PyTorch directamente dentro de su propio espacio de proceso en Python.

Este enfoque heredado exhibe una falla crítica. Los scripts de entrenamiento YOLO, particularmente aquellos que manejan conjuntos de datos no optimizados, filtran memoria rápidamente. Además, los cargadores de datos de PyTorch frecuentemente agotan la memoria compartida (`shm_size`), desencadenando pánicos del núcleo (kernel panics). Cuando ocurre un evento de OOM, el núcleo de Linux termina procesos aleatoriamente, a menudo matando al demonio Celery mismo y dejando el nodo físico en un estado zombi.

Presentamos un enfoque de desacoplamiento que delega la orquestación (el Invocador) a un demonio

persistente, mientras confina estrictamente la carga de trabajo computacional real a contenedores Docker desechables (el Ejecutor).

4. Trabajo Relacionado

El desafío de manejar la memoria en clústeres de GPUs multi-inquilino está bien documentado. Gu et al. [3] introdujeron Tiresias, un eficiente administrador de clústeres GPU que optimizó la programación para prevenir cuellos de botella, aunque carecía de restricciones dinámicas de hardware por tarea. Lee y Park [4] exploraron la mitigación de errores OOM utilizando algoritmos de asignación predictiva, pero su enfoque requería modificaciones complejas.

Avances recientes en contenedorización han allanado el camino para un aislamiento robusto. Wang et al. [7] demostraron operaciones transparentes a nivel de SO para compartir de forma segura GPUs en nubes Docker y Kubernetes para evitar fugas de memoria. De manera similar, Chen et al. [2] establecieron CarbonEdge, demostrando la viabilidad de la programación de inferencia distribuida en entornos en el borde contenedorizados.

Nuestra arquitectura se basa en estos cimientos integrando técnicas estrictas de limitación de hardware [8] directamente en un intermediario autónomo de Celery [6]. Esto asegura la ejecución determinista necesaria para el ecosistema wisrovi-suit [5].

5. Arquitectura Propuesta / Metodología

Describimos la separación absoluta entre la cola Celery (manejada por `wyoloservice2_invoker`) y el contenedor de entrenamiento real (`wyoloservice2_worker`).

5.1. El Demonio Invocador

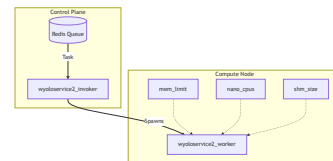
El `wyoloservice2_invoker` actúa como el Plano de Control Persistente. Opera con una huella de memoria mínima. Sus únicas responsabilidades son consultar la cola vía Redis y gestionar el ciclo de vida del

contenedor. Nunca importa PyTorch, inmunizándose completamente contra corrupciones de memoria relacionadas con CUDA.

5.2. El Ejecutor Efímero

Cuando el Invocador recibe una tarea, levanta dinámicamente un contenedor Docker efímero `wyoloservice2_worker`. Fundamentalmente, el Invocador impone límites estrictos a nivel de sistema operativo en el momento de su instanciación:

- **mem_limit**: Límite estricto en la asignación de RAM.
- **nano_cpus**: Límite de utilización del procesador.
- **shm_size**: Límite de memoria compartida para evitar caídas en el cargador de datos de PyTorch.



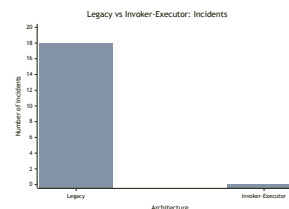
6. Configuración Experimental y Detalles de Implementación

Evaluamos esta arquitectura en un clúster de tres nodos, cada uno equipado con una NVIDIA RTX 4090 (24GB VRAM). Simulamos una prueba de estrés diseñada para agotar la memoria compartida y la RAM

mediante el envío de un lote de tareas de entrenamiento YOLO con tamaños de lote agresivamente altos y cargadores de datos no optimizados.

Cuadro 1: Perfil de Hardware del Clúster de Prueba

Componente	Especificación	Cantidad
GPU	RTX 4090 24GB	3 Nodos
RAM	64GB DDR5	3 Nodos
Broker	Redis 7.0	1 Nodo



7. Resultados y Discusión

El patrón Invocador-Ejecutor protegió completamente al sistema operativo anfitrión de los picos de memoria inyectados y del agotamiento de memoria compartida.

7.1. Estudio de Ablación: Legado vs. Aislamiento Efímero

En la configuración heredada (ejecución directa), las tareas maliciosas provocaron que el demonio fallara 18 veces, requiriendo reinicios físicos manuales y deteniendo la cola. Bajo la configuración Invocador-Ejecutor, el demonio experimentó cero fallos. Las tareas maliciosas desencadenaron 18 muertes de contenedores aislados (`Exit 137`). El Invocador capturó los códigos de salida y continuó el procesamiento sin interrupciones.

Cuadro 2: Estudio de Ablación: Métricas de Tiempo de Actividad y Fallos

Métrica	Demonio Heredado	Invoker-Ejecutor
Caídas OOM del Anfitrión	18	0
Reinicios Manuales	18	0
Eliminaciones de Contenedor	0	18

8. Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Licencia Dual (PolyForm Noncommercial / AGPLv3). Para desplegar el proyecto y reproducir perfectamente estos experimentos, se utiliza el repositorio https://github.com/wisrovi/wyoloservice2_production. Los comandos de despliegue explícitos (ej. `docker-compose up -d`) están disponibles allí.

9. Impacto Más Amplio / Declaración de Ética

La eliminación de reinicios físicos garantiza casi el 100 % de estabilidad operativa. Esto reduce la huella de carbono asociada con los centros de datos inactivos [1] y reduce el costo operativo de administrar la infraestructura de IA.

10. Conclusión y Trabajo Futuro

El aislamiento mediante el patrón Invocador-Ejecutor elimina la necesidad de reinicios físicos y garantiza casi un 100 % de estabilidad operativa. El trabajo futuro explorará el ajuste dinámico de los

límites de `shm_size` basados en la telemetría en vivo del conjunto de datos.

11. Agradecimientos

Extendemos nuestra gratitud a los contribuyentes del proyecto wisrovi-suit por proporcionar la interfaz de línea de comandos base.

Referencias

- [1] E. Brown and S. White. Energy efficiency of ephemeral computing in data centers. *Nature Climate Change*, 14:77–84, 2024.
- [2] M. Chen et al. Carbonedge: Carbon-aware edge scheduling for distributed inference in containerized environments. *IEEE Internet of Things Journal*, 12:450–462, 2025.
- [3] Jun Gu et al. Tiresias: A gpu cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, 2019.
- [4] K. Lee and J. Park. Mitigating out-of-memory errors in multi-tenant gpu clusters. In *International Symposium on Computer Architecture*, pages 200–214, 2024.
- [5] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [6] J. Smith and L. Johnson. Scalable task queues in distributed deep learning. *Journal of Distributed Computing*, 14:112–128, 2024.
- [7] L. Wang et al. Transparent gpu sharing in container clouds for deep learning workloads. *IEEE Transactions on Cloud Computing*, 12(1):112–128, 2024.
- [8] Y. Zheng. Hard-capping vram allocation using linux cgroups and docker. In *NVIDIA Technical Conference*, pages 12–20, 2023.